Train/Test Split

La elegante técnica de dividir datos solo para confirmar que tus errores no necesitan más entrenamiento: se reproducen solos

Daniel Barajas Higuera

MLEARN

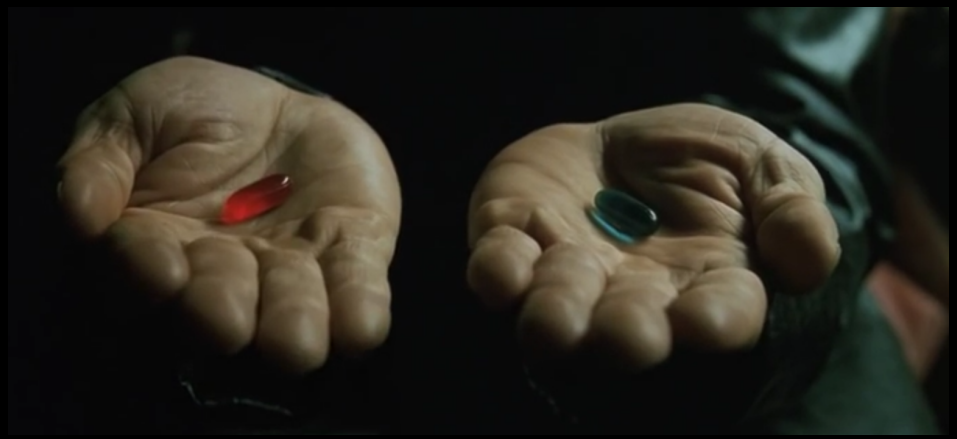


Contenido

1. Introducción
2. Entrenamiento y prueba
3. Entrenamiento, Validación y Prueba

¿Entrenar sin probar?

¿Confiarías en un modelo que solo ha sido entrenado, pero nunca puesto a prueba?

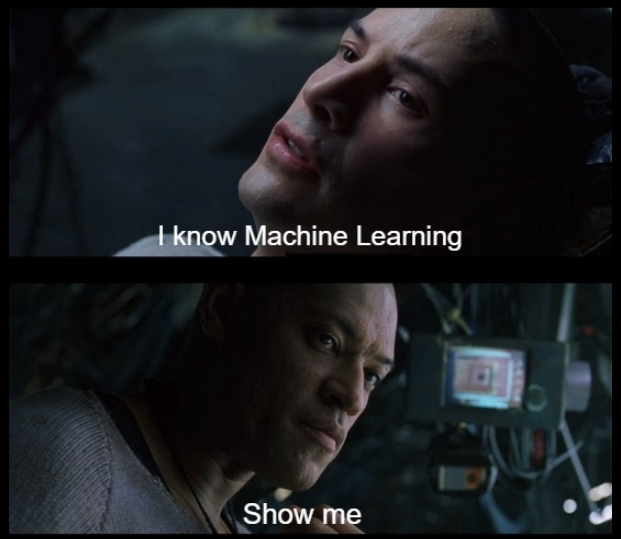


- En *The Matrix*, Neo debe elegir: seguir en su zona de confort o enfrentarse a la realidad.
- En machine learning, también debemos decidir: ¿cómo saber si un modelo funciona realmente?

¿Por qué separar datos?

Separar los datos en distintos conjuntos nos permite evaluar qué tan bien funciona un modelo con información que no ha visto antes. Si usamos todos los datos solo para entrenar, no sabemos si el modelo realmente aprendió patrones útiles o simplemente memorizó.

- Entrenar y probar con los mismos datos es engañoso.
- Necesitamos saber si el modelo generaliza.
- Por eso separamos datos para entrenamiento y evaluación.



Contenido

1 Introducción

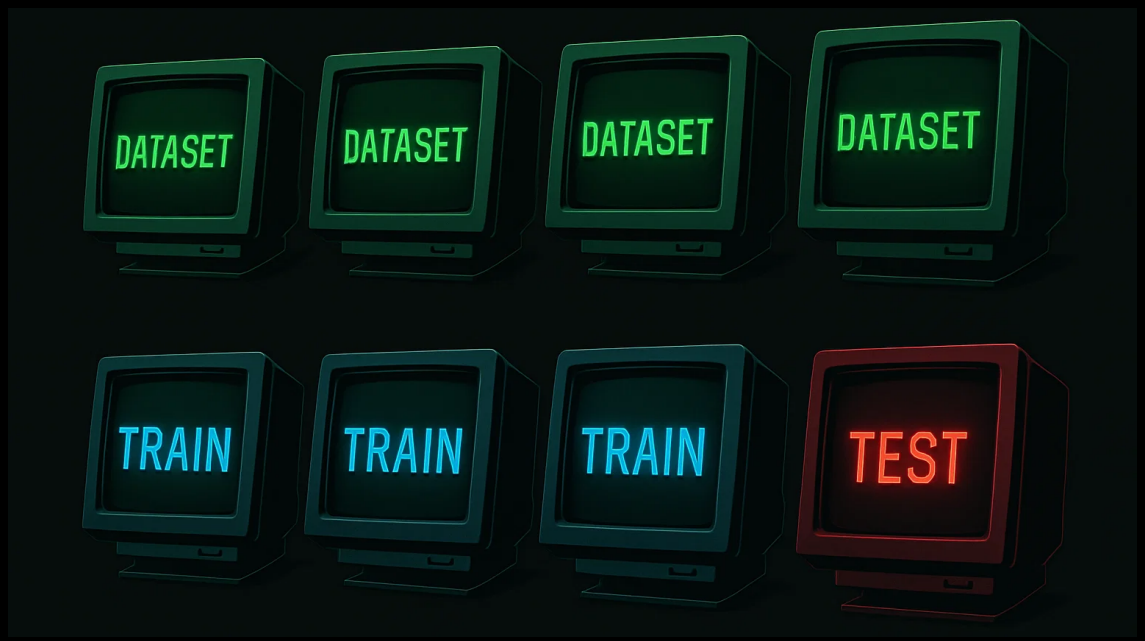
2 Entrenamiento y prueba

3 Entrenamiento, Validación y Prueba

Entrenamiento y prueba

El enfoque más simple consiste en dividir el conjunto total en dos partes: una para entrenar el modelo y otra para probar qué tan bien funciona. Esta separación permite detectar si el modelo se está sobreajustando a los datos de entrenamiento.

- 70 % para entrenamiento: el modelo aprende.
- 30 % para prueba: medimos su rendimiento.
- El conjunto de prueba debe mantenerse separado.



Ejemplo Entrenamiento

Neo entrena en la Matrix con programas de combate y simulaciones. Todo eso representa el conjunto de entrenamiento.

- Aprendizaje en entorno controlado.
- Pruebas repetitivas y guiadas.
- Objetivo: adquirir habilidades básicas.



Entrenamiento y prueba

```
1  import seaborn as sns
2  import pandas as pd
3  from sklearn.model_selection import train_test_split
4  from sklearn.linear_model import LinearRegression
5  from sklearn.metrics import mean_squared_error
6
7  df = sns.load_dataset('diamonds')
8  X = df[['carat', 'depth', 'table','volume']]
9  y = df['price']
10
11 # Continúa ...
```

Entrenamiento y prueba (cont)

```
1
2   # ...
3   # Dividir en entrenamiento (70%) y prueba (30%)
4   X_train, X_test, y_train, y_test = train_test_split(X, y,
5                                                       test_size=0.2)
6
7   modelo = LinearRegression()      # modelo lineal simple
8   modelo.fit(X_train, y_train)
9   y_pred = modelo.predict(X_test) # predicción
10
11  rmse = mean_squared_error(y_test, y_pred, squared=False)
12  print(f"RMSE en conjunto de prueba: {rmse:.2f}")
```

Entrenamiento y prueba

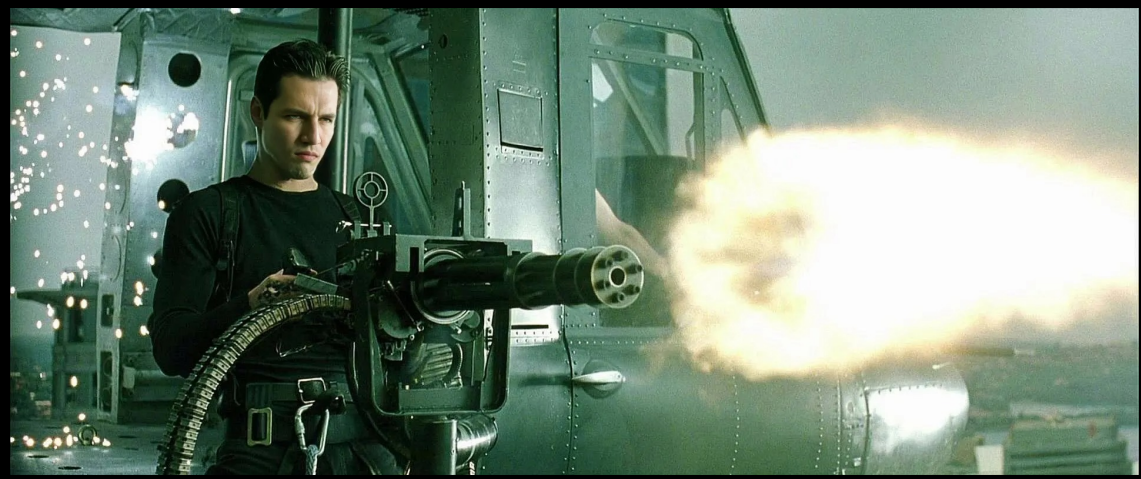
```
# Dividir en entrenamiento (80%) y prueba (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2)
```

- `train_test_split` divide los datos en dos conjuntos: entrenamiento y prueba.
- Devuelve una tupla con 4 elementos:
  - `X_train`, `y_train`: características y precios para entrenar el modelo.
  - `X_test`, `y_test`: características y precios para evaluar el modelo.
- `test_size=0.2` indica que el 20 % de los datos se usan para prueba y el 80 % para entrenamiento.
- `X` contiene los predictores, `y` contiene la variable objetivo (`price`).

Ejemplo Prueba

La verdadera prueba ocurre fuera de la Matrix, donde Neo debe usar lo aprendido en situaciones no simuladas.

- Escenarios impredecibles.
- Sin asistencia directa.
- Se evalúa si el "modelo Neo" generaliza.



Contenido

1 Introducción

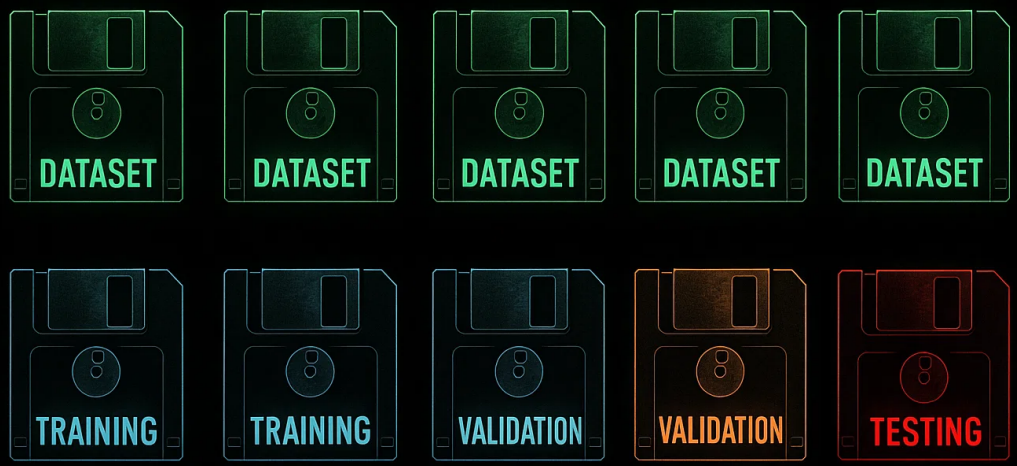
2 Entrenamiento y prueba

3 Entrenamiento, Validación y Prueba

Entrenamiento, Validación y Prueba

Al entrenar modelos más complejos, necesitamos ajustar sus parámetros (por ejemplo, el número de neuronas). Para eso usamos un conjunto de validación. La idea es no tocar el conjunto de prueba hasta el final.

- Entrenamiento: el modelo aprende.
- Validación: se ajustan parámetros.
- Prueba: se mide el rendimiento final.



Entrenamiento en Matrix

Aunque introducimos una etapa de validación, la lógica del entrenamiento no cambia.



Entrenamiento en Matrix

- El modelo sigue aprendiendo a partir del conjunto de entrenamiento.
- Se ajustan los pesos o parámetros internos con esos datos.



Validación en Matrix: El salto

Todos fallan la primera vez. El salto es una prueba de fe y convicción más que de habilidad.



Validación en Matrix: El salto

- El entrenamiento previo no garantiza éxito.
- Esta validación revela que aún no cree del todo en la simulación.
- Es un punto de quiebre que muestra qué tanto ha interiorizado el modelo.



Entrenamiento, validación y prueba

```
1  import seaborn as sns
2  import pandas as pd
3  from sklearn.model_selection import train_test_split
4  from sklearn.linear_model import LinearRegression
5  from sklearn.metrics import mean_squared_error
6
7  df = sns.load_dataset('diamonds')
8
9  X = df[['carat', 'depth', 'table','volume']]
10 y = df['price']
11
12 # Separar en entrenamiento (60%), validación (20%) y prueba (20%)
13 X_temp, X_test, y_temp, y_test = train_test_split(X, y,
14                                                  test_size=0.2)
15
16 X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp,
17                                                  test_size=0.25)
18 # Continua ...
19
```

Entrenamiento, validación y prueba

```
X_temp, X_test, y_temp, y_test = train_test_split(X, y,
                                                  test_size=0.2)

X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp,
                                                  test_size=0.25)

modelo = LinearRegression()
modelo.fit(X_train, y_train)

# Validación
y_val_pred = modelo.predict(X_val)
rmse_val = mean_squared_error(y_val, y_val_pred, squared=False)

# Prueba final
y_test_pred = modelo.predict(X_test)
rmse_test = mean_squared_error(y_test, y_test_pred,
↪  squared=False)

print(f"RMSE validación: {rmse_val:.2f}")
print(f"RMSE prueba final: {rmse_test:.2f}")
```

Entrenamiento, validación y prueba

```
# Separar en entrenamiento (60%), validación (20%) y prueba (20%)
X_temp, X_test, y_temp, y_test = train_test_split(X, y,
↪  test_size=0.2)
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp,
↪  test_size=0.25)
```

- Se hace una primera división: 80 % entrenamiento+validación y 20 % prueba.
- Luego se divide el 80 % en:
  - 75 % entrenamiento $\Rightarrow$ 60 % total
  - 25 % validación $\Rightarrow$ 20 % total
- Esto permite ajustar el modelo y evaluarlo en validación antes de medir desempeño final en prueba.
- `train_test_split` se usa dos veces para lograr esto.

Validación en Matrix: El Oráculo

Después de entrenar, Neo se encuentra con el Oráculo. Ella no le da respuestas claras, sino que lo enfrenta a dilemas que desafían su identidad y propósito.



Validación en Matrix: El Oráculo

- Le hace dudar, no lo juzga con pruebas físicas.
- Esta validación es emocional y filosófica.
- Es el paso previo antes de ser puesto a prueba en el mundo real.



Prueba en Matrix: Enfrentando a Smith

Llega el momento de aplicar todo lo aprendido. Ya no hay entorno controlado, ni consejos. Solo acción y consecuencias reales.

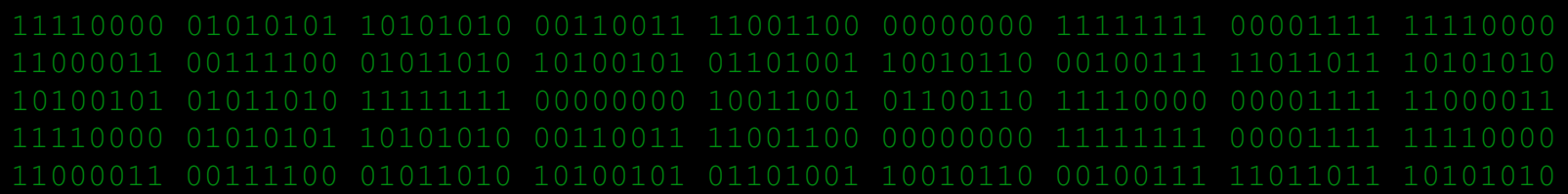


Prueba en Matrix: Enfrentando a Smith

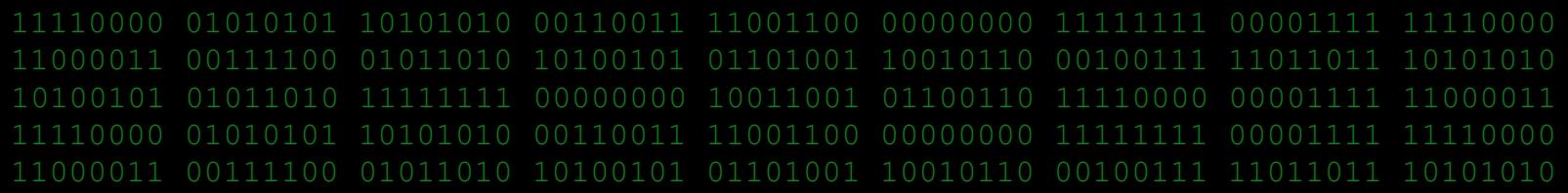
- Es el conjunto de prueba: mide el desempeño final.
- No se ajustan más parámetros, se mide el resultado.
- O superas al modelo adversario, o fallas.



Comparación de enfoques



Método	Entrenamiento	Validación	Prueba
Train-Test	70 %	–	30 %
Train-Val-Test	60 %	20 %	20 %



A tener en cuenta

- "Dividir los datos no es un ritual, es la única forma de saber si tu modelo es real."
- "Validar y probar revela si aprendió... o si solo repite lo que ya vio."
- "En el mundo fuera del aula, separar los datos decide si tu modelo despierta... o sigue atrapado en la Matrix."

